

API Endpoints: Implementation Guide

Introduction

This how-to guide explains the process of implementing basic API endpoints (GET, POST, PUT, DELETE) in an ASP.NET Core project.

Steps

1. Set Up the Development Environment
 - a. Download and install the .NET SDK from Microsoft's official site.
 - b. Install Visual Studio Code and ensure the ASP.NET and web development workload is selected during installation.
2. Create a New ASP.NET Core Web API Project
 - a. Open Command Prompt or Terminal.
 - b. Run this command to create a new project:
3. Add a Controller
 - a. In Visual Studio Code, add a new controller class (e.g., WeatherForecastController.cs).
 - b. Add these attributes to the class:

```
dotnet new webapi -n MyWebAPI
```

- c. This generates a new Web API project with the basic folder structure.

```
public class WeatherForecast
```

```
{
```

```
    public DateTime Date { get; set; }
```

```
    public int TemperatureC { get; set; }
```

```
    public string? Summary { get; set; }
```

```
}
```

```
[ApiController]
```

```
[Route("[controller]")]
```

```
public class WeatherForecastController : ControllerBase
```

```

{

    private static readonly string[] Summaries = new[]

    {

        "Freezing", "Bracing", "Chilly", "Cool", "Mild", "Warm",
        "Balmy", "Hot", "Sweltering", "Scorching"

    };

    // Method implementations go here

}

```

4. Define the API Methods

- a. GET Method (Retrieve Data) - This method returns a list of weather forecasts.

```

[HttpGet]
public IEnumerable<WeatherForecast> Get()

{

    var rng = new Random();

    return Enumerable.Range(1, 5).Select(index => new
WeatherForecast

    {

        Date = DateTime.Now.AddDays(index),

        TemperatureC = rng.Next(-20, 55),

        Summary = Summaries[rng.Next(Summaries.Length)]

    }).ToArray();

}

```

- b. POST Method (Create Data) - This method accepts a data object in the request body and returns the created object.

```

[HttpPost]
public IActionResult Post([FromBody] WeatherForecast forecast)

{

    // Add data to storage (e.g., database)

    return Ok(forecast);

}

```

- c. PUT Method (Update Data) - This method updates an existing item based on its ID.

```
[HttpPut("{id}")]
```

```
public IActionResult Put(int id, [FromBody] WeatherForecast
forecast)
{
    // Update data for the given ID
    // Example: Find and update an item with a matching ID
    var existingForecast = /* fetch the data */;
    existingForecast.Date = forecast.Date
    return NoContent();
}
```

- d. DELETE Method (Remove Data) - This method deletes an item based on its ID.

```
[HttpDelete("{id}")]
```

```
public IActionResult Delete(int id)
{
    // Delete data for the given ID
    return NoContent();
}
```

5. Run the Application

- a. Open Command Prompt or Terminal
- b. Change directory if you haven't already with `cd MyWebAPI`
- c. Run the project using `dotnet run`

Access the API using a browser or API testing tool (like Postman) by navigating to: Weather Forecast Endpoint at: <http://localhost:5215/WeatherForecast>

Note: your port may not be 5215. To find the port which the API is using, you can either check the terminal output for "Now listening on: <http://localhost:<your-port>>", or you can go into the Properties folder and open the launchSettings.json file to find the port number.

Now you have a functional API with basic GET, POST, PUT, and DELETE endpoints using ASP.NET Core. You can test and expand these methods based on your project needs.